Razonamiento sobre programas

Tema 5 — Razonamiento Automático

Fabián Fernando Serrano Suárez

Isabelle/HOL

Ciencias de la Computación

Recorrido

Razonamiento ecuacional

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Inducción según la definición

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Inducción según la definición

Razonamiento por casos

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Inducción según la definición

Razonamiento por casos

Heurística de generalización

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Inducción según la definición

Razonamiento por casos

Heurística de generalización

Funciones de orden superior

Recorrido

Razonamiento ecuacional

Inducción sobre los naturales

Inducción sobre listas

Inducción según la definición

Razonamiento por casos

Heurística de generalización

Funciones de orden superior

Resumen

Razonamiento ecuacional

Probar un caso no es demostrar

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

lemma "longitud [4,2,5] = 3"
by simp
```

Probar un caso no es demostrar

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

lemma "longitud [4,2,5] = 3"
by simp
```

- Esto es una demostración, pero de un caso concreto: es lo mismo que hacer `value`.

Probar un caso no es demostrar

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

lemma "longitud [4,2,5] = 3"
by simp
```

- Esto es una demostración, pero de un caso concreto: es lo mismo que hacer `value`.
- Lo interesante son las propiedades que valen para todas las entradas: `longitud (xs @ ys) = ...` para cualquier `xs` y `ys`.

Probar un caso no es demostrar

```
fun longitud :: "'a list ⇒ nat" where
  "longitud []     = 0"
| "longitud (x#xs) = 1 + longitud xs"

lemma "longitud [4,2,5] = 3"
by simp
```

- Esto es una demostración, pero de un caso concreto: es lo mismo que hacer `value`.
- Lo interesante son las propiedades que valen para todas las entradas: `longitud (xs @ ys) = ...` para cualquier `xs` y `ys`.

Ese es el tema de hoy: demostrar que un programa hace lo que debe, para todas las entradas.

Ejemplo 4: intercambia (intercambia (x,y)) = (x,y)

```
fun intercambia :: "'a × 'b ⇒ 'b × 'a" where
  "intercambia (x,y) = (y,x)"

lemma "intercambia (intercambia (x,y)) = (x,y)"
proof -
  have "intercambia (intercambia (x,y)) = intercambia (y,x)"
    by (simp only: intercambia.simps)
  also have "... = (x,y)"
    by (simp only: intercambia.simps)
  finally show "intercambia (intercambia (x,y)) = (x,y)" by simp
qed
```

Ejemplo 4: intercambia (intercambia (x,y)) = (x,y)

```
fun intercambia :: "'a × 'b ⇒ 'b × 'a" where
  "intercambia (x,y) = (y,x)"

lemma "intercambia (intercambia (x,y)) = (x,y)"
proof -
  have "intercambia (intercambia (x,y)) = intercambia (y,x)"
    by (simp only: intercambia.simps)
  also have "... = (x,y)"
    by (simp only: intercambia.simps)
  finally show "intercambia (intercambia (x,y)) = (x,y)" by simp
qed
```

- `also... finally`: una cadena de igualdades, como en papel. `...` es el lado derecho de la igualdad anterior.

Ejemplo 4: intercambia (intercambia (x,y)) = (x,y)

```
fun intercambia :: "'a × 'b ⇒ 'b × 'a" where
  "intercambia (x,y) = (y,x)"

lemma "intercambia (intercambia (x,y)) = (x,y)"
proof -
  have "intercambia (intercambia (x,y)) = intercambia (y,x)"
    by (simp only: intercambia.simps)
  also have "... = (x,y)"
    by (simp only: intercambia.simps)
  finally show "intercambia (intercambia (x,y)) = (x,y)" by simp
qed
```

- `also... finally`: una cadena de igualdades, como en papel. `...` es el lado derecho de la igualdad anterior.
- `intercambia.simps` son las ecuaciones de la definición: cada paso se justifica con una de ellas.

Ejemplo 6: inversa [x] = [x]

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

Ejemplo 6: `inversa [x] = [x]`

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- `[x]` abrevia `x#[]`.

Ejemplo 6: inversa [x] = [x]

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- `[x]` abrevia `x#[]`.
- Se aplica la segunda ecuación de `inversa`: `inversa.simps(2)`.

Ejemplo 6: `inversa [x] = [x]`

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- `[x]` abrevia `x#[]`.
- Se aplica la segunda ecuación de `inversa`: `inversa.simps(2)`.
- Luego la primera, para `inversa []`.

Ejemplo 6: `inversa [x] = [x]`

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- `[x]` abrevia `x#[]`.
- Se aplica la segunda ecuación de `inversa`: `inversa.simps(2)`.
- Luego la primera, para `inversa []`.
- `append_Nil` es un teorema de la biblioteca: `[] @ ys = ys`.

Ejemplo 6: `inversa [x] = [x]`

```
fun inversa :: "'a list ⇒ 'a list" where
  "inversa []     = []"
| "inversa (x#xs) = inversa xs @ [x]"

lemma "inversa [x] = [x]"
proof -
  have "inversa [x] = inversa (x#[])" by simp
  also have "... = (inversa []) @ [x]"
    by (simp only: inversa.simps(2))
  also have "... = [] @ [x]"
    by (simp only: inversa.simps(1))
  also have "... = [x]"
    by (simp only: append_Nil)
  finally show "inversa [x] = [x]" by simp
qed
```

- `[x]` abrevia `x#[]`.
- Se aplica la segunda ecuación de `inversa`: `inversa.simps(2)`.
- Luego la primera, para `inversa []`.
- `append_Nil` es un teorema de la biblioteca: `[] @ ys = ys`.

Con `by simp` a secas sale en una línea. Escribirla así muestra qué hace `simp` por dentro.

Inducción sobre los naturales

El principio de inducción

Para demostrar P para todos los naturales, basta probar que el `0` la cumple, y que si `n` la cumple también la cumple `Suc n`.

El principio de inducción

Para demostrar P para todos los naturales, basta probar que el `0` la cumple, y que si `n` la cumple también la cumple `Suc n`.

```
nat.induct
```

$$\llbracket \texttt{P 0};\ \bigwedge \texttt{n. P n} \Longrightarrow \texttt{P (Suc n)} \rrbracket \Longrightarrow \texttt{P m}$$

El principio de inducción

Para demostrar P para todos los naturales, basta probar que el `0` la cumple, y que si `n` la cumple también la cumple `Suc n`.

```
nat.induct
```

$$\llbracket \texttt{P 0};\ \bigwedge \texttt{n. P n} \Longrightarrow \texttt{P (Suc n)} \rrbracket \Longrightarrow \texttt{P m}$$

	Definición recursiva	**Demostración por inducción**
`0`	`suma 0 = 0`	caso base: `P 0`
`Suc n`	`suma (Suc m) = ... suma m`	paso: de `P n`, `P (Suc n)`

El principio de inducción

Para demostrar P para todos los naturales, basta probar que el `0` la cumple, y que si `n` la cumple también la cumple `Suc n`.

```
nat.induct
```

```
⟦P 0; ⋀n. P n ⟹ P (Suc n)⟧ ⟹ P m
```

	Definición recursiva	**Demostración por inducción**
`0`	`suma 0 = 0`	caso base: `P 0`
`Suc n`	`suma (Suc m) = ... suma m`	paso: de `P n`, `P (Suc n)`

Recursión e inducción tienen la misma forma: una para calcular, otra para demostrar.

Ejemplo 8: longitud (repite n x) = n

```
fun repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"

lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) =
          longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n"
    by simp
qed
```

Ejemplo 8: `longitud (repite n x) = n`

```
fun repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"

lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) =
          longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n"
    by simp
qed
```

- `proof (induct n)` aplica **nat.induct**: quedan dos casos, separados por `next`.

Ejemplo 8: `longitud (repite n x) = n`

```
fun repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"

lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) =
          longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n"
    by simp
qed
```

- `proof (induct n)` aplica **nat.induct**: quedan dos casos, separados por `next`.
- **Caso base:** `n = 0`, sale por simplificación.

Ejemplo 8: `longitud (repite n x) = n`

```
fun repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"

lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) =
          longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n"
    by simp
qed
```

- `proof (induct n)` aplica **nat.induct**: quedan dos casos, separados por `next`.
- **Caso base:** `n = 0`, sale por simplificación.
- **Paso:** `fix n`, «sea `n` arbitrario», y se supone la propiedad para `n`: es la hipótesis de inducción, `HI`.

Ejemplo 8: `longitud (repite n x) = n`

```
fun repite :: "nat ⇒ 'a ⇒ 'a list" where
  "repite 0 x       = []"
| "repite (Suc n) x = x # (repite n x)"

lemma "longitud (repite n x) = n"
proof (induct n)
  show "longitud (repite 0 x) = 0" by simp
next
  fix n
  assume HI: "longitud (repite n x) = n"
  have "longitud (repite (Suc n) x) =
          longitud (x # (repite n x))" by simp
  also have "... = 1 + longitud (repite n x)" by simp
  also have "... = 1 + n" using HI by simp
  finally show "longitud (repite (Suc n) x) = Suc n"
    by simp
qed
```

- `proof (induct n)` aplica **nat.induct**: quedan dos casos, separados por `next`.
- **Caso base:** `n = 0`, sale por simplificación.
- **Paso:** `fix n`, «sea `n` arbitrario», y se supone la propiedad para `n`: es la hipótesis de inducción, `HI`.
- La cadena de igualdades usa `HI` en un solo paso: el que pasa de `repite n x` a `n`.

La misma demostración, automática

```
lemma "longitud (repite n x) = n"
by (induct n) auto
```

La misma demostración, automática

```
lemma "longitud (repite n x) = n"
by (induct n) auto
```

- `induct n` plantea los dos casos; `auto` cierra ambos.

La misma demostración, automática

```
lemma "longitud (repite n x) = n"
by (induct n) auto
```

- `induct n` plantea los dos casos; `auto` cierra ambos.
- Lo que no puede hacer solo es decidir sobre qué hacer inducción. Eso lo elige usted.

La misma demostración, automática

```
lemma "longitud (repite n x) = n"
by (induct n) auto
```

- `induct n` plantea los dos casos; `auto` cierra ambos.
- Lo que no puede hacer solo es decidir sobre qué hacer inducción. Eso lo elige usted.

Escriba primero la versión estructurada: cuando `auto` falle, es la única forma de ver en qué caso y en qué paso se atasca.

Inducción sobre listas

El principio de inducción sobre listas

Para demostrar P para todas las listas, basta probar que `[]` la cumple, y que si `xs` la cumple también la cumple `x#xs`.

El principio de inducción sobre listas

Para demostrar P para todas las listas, basta probar que `[]` la cumple, y que si `xs` la cumple también la cumple `x#xs`.

```
list.induct
```

$$\llbracket \texttt{P []};\ \bigwedge \texttt{x xs. P xs} \Longrightarrow \texttt{P (x\#xs)} \rrbracket \Longrightarrow \texttt{P ys}$$

El principio de inducción sobre listas

Para demostrar P para todas las listas, basta probar que `[]` la cumple, y que si `xs` la cumple también la cumple `x#xs`.

```
list.induct
```

$$\llbracket \texttt{P []};\ \bigwedge \texttt{x xs. P xs} \Longrightarrow \texttt{P (x\#xs)} \rrbracket \Longrightarrow \texttt{P ys}$$

Es la misma idea que con los naturales: `[]` hace el papel de `0`, y `x#xs` el de `Suc n`.

Un caso por constructor del tipo de datos.

Ejemplo 10: conc es asociativa

Con `conc [] ys = ys` y `conc (x#xs) ys = x # (conc xs ys)`:

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs"
    by simp
qed
```

Ejemplo 10: conc es asociativa

Con `conc [] ys = ys` y `conc (x#xs) ys = x # (conc xs ys)`:

```
lemma "conc xs (conc ys zs) = conc (conc xs ys) zs"
proof (induct xs)
  show "conc [] (conc ys zs) = conc (conc [] ys) zs" by simp
next
  fix x xs
  assume HI: "conc xs (conc ys zs) = conc (conc xs ys) zs"
  have "conc (x # xs) (conc ys zs) = x # (conc xs (conc ys zs))" by simp
  also have "... = x # (conc (conc xs ys) zs)" using HI by simp
  also have "... = conc (conc (x # xs) ys) zs" by simp
  finally show "conc (x # xs) (conc ys zs) = conc (conc (x # xs) ys) zs"
    by simp
qed
```

Se induce sobre xs, el argumento en el que conc hace recursión. Se induce sobre la variable en la que recursa la función.

Ejemplo 11: refutar antes de demostrar

¿Es conc conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

Ejemplo 11: refutar antes de demostrar

¿Es `conc` conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores.

Ejemplo 11: refutar antes de demostrar

¿Es `conc` conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores.
- Lo encuentra enseguida: dos listas de un elemento, distintas. `conc [a] [b] = [a,b]`, pero `conc [b] [a] = [b,a]`.

Ejemplo 11: refutar antes de demostrar

¿Es `conc` conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores.
- Lo encuentra enseguida: dos listas de un elemento, distintas. `conc [a] [b] = [a,b]`, pero `conc [b] [a] = [b,a]`.
- `oops` abandona el lema: es falso, no hay nada que demostrar.

Ejemplo 11: refutar antes de demostrar

¿Es `conc` conmutativa?

```
lemma "conc xs ys = conc ys xs"
quickcheck
oops
```

- `quickcheck` busca un contraejemplo probando valores.
- Lo encuentra enseguida: dos listas de un elemento, distintas. `conc [a] [b] = [a,b]`, pero `conc [b] [a] = [b,a]`.
- `oops` abandona el lema: es falso, no hay nada que demostrar.

Hábito de trabajo: antes de pasar una hora intentando demostrar algo, pregúntele a `quickcheck` si es verdad.

Inducción según la definición

Funciones con más de dos ecuaciones

```
fun coge :: "nat ⇒ 'a list ⇒ 'a list" where
  "coge n []           = []"
| "coge 0 xs           = []"
| "coge (Suc n) (x#xs) = x # (coge n xs)"
```

Funciones con más de dos ecuaciones

```
fun coge :: "nat ⇒ 'a list ⇒ 'a list" where
  "coge n []           = []"
| "coge 0 xs           = []"
| "coge (Suc n) (x#xs) = x # (coge n xs)"
```

```
value "coge 2 [a,c,d,b,e]"
```
⇝ [a, c] y elimina 2 [a,c,d,b,e] ⇝ [d, b, e]

Funciones con más de dos ecuaciones

```
fun coge :: "nat ⇒ 'a list ⇒ 'a list" where
  "coge n []           = []"
| "coge 0 xs           = []"
| "coge (Suc n) (x#xs) = x # (coge n xs)"
```

```
value "coge 2 [a,c,d,b,e]"
```
⇝ `[a, c]` y `elimina 2 [a,c,d,b,e]` ⇝ `[d, b, e]`

`coge.induct`: **Isabelle lo genera a partir de la definición**

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

Funciones con más de dos ecuaciones

```
fun coge :: "nat ⇒ 'a list ⇒ 'a list" where
  "coge n []           = []"
| "coge 0 xs           = []"
| "coge (Suc n) (x#xs) = x # (coge n xs)"
```

`value "coge 2 [a,c,d,b,e]"` ⇝ `[a, c]` y `elimina 2 [a,c,d,b,e]` ⇝ `[d, b, e]`

`coge.induct`: Isabelle lo genera a partir de la definición

```
⟦⋀n. P n [];
 ⋀x xs. P 0 (x#xs);
 ⋀n x xs. P n xs ⟹ P (Suc n) (x#xs)⟧ ⟹ P n xs
```

Un caso por ecuación. La inducción sigue exactamente la forma de la recursión, aunque recorra dos argumentos a la vez.

Ejemplo 16: conc (coge n xs) (elimina n xs) = xs

```
lemma "conc (coge n xs)
           (elimina n xs) = xs"
proof (induct rule: coge.induct)
  fix n
  show "conc (coge n [])
             (elimina n []) = []"
    by simp
next
  fix x xs
  show "conc (coge 0 (x#xs))
             (elimina 0 (x#xs))
         = x#xs" by simp
next
  fix n x xs
  assume HI: "conc (coge n xs)
                   (elimina n xs) = xs"
  have "conc (coge (Suc n) (x#xs))
             (elimina (Suc n) (x#xs)) =
       conc (x#(coge n xs)) (elimina n xs)"
    by simp
  also have "... = x#(conc (coge n xs)
                           (elimina n xs))"
    by simp
  also have "... = x#xs" using HI by simp
  finally show "conc (coge (Suc n) (x#xs))
                     (elimina (Suc n) (x#xs))
                = x#xs" by simp
qed
```

Ejemplo 16: conc (coge n xs) (elimina n xs) = xs

```
lemma "conc (coge n xs)
            (elimina n xs) = xs"
proof (induct rule: coge.induct)
  fix n
  show "conc (coge n [])
             (elimina n []) = []"
    by simp
next
  fix x xs
  show "conc (coge 0 (x#xs))
             (elimina 0 (x#xs))
          = x#xs" by simp
next
  fix n x xs
  assume HI: "conc (coge n xs)
                   (elimina n xs) = xs"
  have "conc (coge (Suc n) (x#xs))
             (elimina (Suc n) (x#xs)) =
        conc (x#(coge n xs)) (elimina n xs)"
    by simp
  also have "... = x#(conc (coge n xs)
                           (elimina n xs))"
    by simp
  also have "... = x#xs" using HI by simp
  finally show "conc (coge (Suc n) (x#xs))
                     (elimina (Suc n) (x#xs))
                = x#xs" by simp
qed
```

Tres casos, uno por ecuación de coge.

Ejemplo 16: conc (coge n xs) (elimina n xs) = xs

```
lemma "conc (coge n xs)
            (elimina n xs) = xs"
proof (induct rule: coge.induct)
  fix n
  show "conc (coge n [])
             (elimina n []) = []"
    by simp
next
  fix x xs
  show "conc (coge 0 (x#xs))
             (elimina 0 (x#xs))
          = x#xs" by simp
next
  fix n x xs
  assume HI: "conc (coge n xs)
                   (elimina n xs) = xs"
  have "conc (coge (Suc n) (x#xs))
             (elimina (Suc n) (x#xs)) =
        conc (x#(coge n xs)) (elimina n xs)"
    by simp
  also have "... = x#(conc (coge n xs)
                           (elimina n xs))"
    by simp
  also have "... = x#xs" using HI by simp
  finally show "conc (coge (Suc n) (x#xs))
                     (elimina (Suc n) (x#xs))
                = x#xs" by simp
qed
```

Tres casos, uno por ecuación de coge. Solo el recursivo (derecha) usa la hipótesis de inducción.

Razonamiento por casos

Ejemplo 18: esVacia xs = esVacia (conc xs xs)

```
fun esVacia :: "'a list ⇒ bool" where
  "esVacia []     = True"
| "esVacia (x#xs) = False"

lemma "esVacia xs = esVacia (conc xs xs)"
proof (cases xs)
  case Nil
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
next
  case Cons
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
qed
```

Ejemplo 18: esVacia xs = esVacia (conc xs xs)

```
fun esVacia :: "'a list ⇒ bool" where
  "esVacia []     = True"
| "esVacia (x#xs) = False"

lemma "esVacia xs = esVacia (conc xs xs)"
proof (cases xs)
  case Nil
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
next
  case Cons
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
qed
```

- cases xs: o bien xs = [], o bien xs = y#ys. Sin hipótesis de inducción.

Ejemplo 18: esVacia xs = esVacia (conc xs xs)

```
fun esVacia :: "'a list ⇒ bool" where
  "esVacia []     = True"
| "esVacia (x#xs) = False"

lemma "esVacia xs = esVacia (conc xs xs)"
proof (cases xs)
  case Nil
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
next
  case Cons
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
qed
```

- `cases xs`: o bien `xs = []`, o bien `xs = y#ys`. Sin hipótesis de inducción.
- `case Nil` y `case Cons` nombran cada caso y añaden su suposición.

Ejemplo 18: `esVacia xs = esVacia (conc xs xs)`

```
fun esVacia :: "'a list ⇒ bool" where
  "esVacia []     = True"
| "esVacia (x#xs) = False"

lemma "esVacia xs = esVacia (conc xs xs)"
proof (cases xs)
  case Nil
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
next
  case Cons
  thus "esVacia xs = esVacia (conc xs xs)"
    by simp
qed
```

- `cases xs`: o bien `xs = []`, o bien `xs = y#ys`. Sin hipótesis de inducción.
- `case Nil` y `case Cons` nombran cada caso y añaden su suposición.
- Basta cuando la propiedad solo depende de la forma de la lista, no de lo que hay dentro.

Heurística de generalización

Invertir con acumulador

```
fun inversaAcAux :: "'a list ⇒ 'a list ⇒ 'a list" where
  "inversaAcAux [] ys     = ys"
| "inversaAcAux (x#xs) ys = inversaAcAux xs (x#ys)"

fun inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs = inversaAcAux xs []"
```

Invertir con acumulador

```
fun inversaAcAux :: "'a list ⇒ 'a list ⇒ 'a list" where
  "inversaAcAux [] ys     = ys"
| "inversaAcAux (x#xs) ys = inversaAcAux xs (x#ys)"

fun inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs = inversaAcAux xs []"

value "inversaAc [a,c,b,e]"                                    ⇝ [e, b, c, a]
```

Invertir con acumulador

```
fun inversaAcAux :: "'a list ⇒ 'a list ⇒ 'a list" where
  "inversaAcAux [] ys     = ys"
| "inversaAcAux (x#xs) ys = inversaAcAux xs (x#ys)"

fun inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs = inversaAcAux xs []"

value "inversaAc [a,c,b,e]"                                    ⇝ [e, b, c, a]
```

- Es la versión recursiva de cola de `inversa`: más eficiente, porque no concatena en cada paso.

Invertir con acumulador

```
fun inversaAcAux :: "'a list ⇒ 'a list ⇒ 'a list" where
  "inversaAcAux [] ys     = ys"
| "inversaAcAux (x#xs) ys = inversaAcAux xs (x#ys)"

fun inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs = inversaAcAux xs []"

value "inversaAc [a,c,b,e]"                                  ⇝ [e, b, c, a]
```

- Es la versión recursiva de cola de `inversa`: más eficiente, porque no concatena en cada paso.
- Queremos demostrar que calcula lo mismo: `inversaAc xs = inversa xs`.

Invertir con acumulador

```
fun inversaAcAux :: "'a list ⇒ 'a list ⇒ 'a list" where
  "inversaAcAux [] ys     = ys"
| "inversaAcAux (x#xs) ys = inversaAcAux xs (x#ys)"

fun inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs = inversaAcAux xs []"

value "inversaAc [a,c,b,e]"                                  ⇝ [e, b, c, a]
```

- Es la versión recursiva de cola de `inversa`: más eficiente, porque no concatena en cada paso.
- Queremos demostrar que calcula lo mismo: `inversaAc xs = inversa xs`.

Una situación muy real: se optimiza un programa, y hay que garantizar que la versión rápida da el mismo resultado que la sencilla.

Ejemplo 20: por qué hay que generalizar

- Se quiere probar `inversaAcAux xs [] = inversa xs`.

Ejemplo 20: por qué hay que generalizar

- Se quiere probar `inversaAcAux xs [] = inversa xs`.
- Pero en la llamada recursiva el segundo argumento cambia: `inversaAcAux xs (x#ys)`. Con `[]` fijo, la hipótesis de inducción no sirve.

Ejemplo 20: por qué hay que generalizar

- Se quiere probar `inversaAcAux xs [] = inversa xs`.
- Pero en la llamada recursiva el segundo argumento cambia: `inversaAcAux xs (x#ys)`. Con `[]` fijo, la hipótesis de inducción no sirve.
- Solución: demostrar algo más fuerte, para cualquier `ys`.

Ejemplo 20: por qué hay que generalizar

- Se quiere probar `inversaAcAux xs [] = inversa xs`.
- Pero en la llamada recursiva el segundo argumento cambia: `inversaAcAux xs (x#ys)`. Con `[]` fijo, la hipótesis de inducción no sirve.
- Solución: demostrar algo más fuerte, para cualquier `ys`.

```
lemma inversaAcAux_es_inversa:
  "inversaAcAux xs ys = (inversa xs) @ ys"
by (induct xs arbitrary: ys) auto

corollary "inversaAc xs = inversa xs"
by (simp add: inversaAcAux_es_inversa)
```

Ejemplo 20: por qué hay que generalizar

- Se quiere probar `inversaAcAux xs [] = inversa xs`.
- Pero en la llamada recursiva el segundo argumento cambia: `inversaAcAux xs (x#ys)`. Con `[]` fijo, la hipótesis de inducción no sirve.
- Solución: demostrar algo más fuerte, para cualquier `ys`.

```
lemma inversaAcAux_es_inversa:
  "inversaAcAux xs ys = (inversa xs) @ ys"
by (induct xs arbitrary: ys) auto

corollary "inversaAc xs = inversa xs"
by (simp add: inversaAcAux_es_inversa)
```

`arbitrary: ys` hace que la hipótesis de inducción valga para todo `ys`. Volverá en el Tema 7: el compilador tampoco se deja demostrar sin generalizar.

Funciones de orden superior

Ejemplo 24: sum y map

```
fun sum :: "nat list ⇒ nat" where
  "sum []     = 0"
| "sum (x#xs) = x + sum xs"

fun map :: "('a ⇒ 'b) ⇒ 'a list ⇒ 'b list" where
  "map f []     = []"
| "map f (x#xs) = (f x) # map f xs"
```

Ejemplo 24: sum y map

```
fun sum :: "nat list ⇒ nat" where
  "sum []     = 0"
| "sum (x#xs) = x + sum xs"

fun map :: "('a ⇒ 'b) ⇒ 'a list ⇒ 'b list" where
  "map f []     = []"
| "map f (x#xs) = (f x) # map f xs"

value "map (λx. 2*x) [3::nat,2,5]"                    ⇝ [6, 4, 10]
```

Ejemplo 24: sum y map

```
fun sum :: "nat list ⇒ nat" where
  "sum []     = 0"
| "sum (x#xs) = x + sum xs"

fun map :: "('a ⇒ 'b) ⇒ 'a list ⇒ 'b list" where
  "map f []     = []"
| "map f (x#xs) = (f x) # map f xs"

value "map (λx. 2*x) [3::nat,2,5]"                    ⇝ [6, 4, 10]

lemma "sum (map (λx. 2*x) xs) = 2 * (sum xs)"
by (induct xs) auto
```

Ejemplo 24: sum y map

```
fun sum :: "nat list ⇒ nat" where
  "sum []     = 0"
| "sum (x#xs) = x + sum xs"

fun map :: "('a ⇒ 'b) ⇒ 'a list ⇒ 'b list" where
  "map f []     = []"
| "map f (x#xs) = (f x) # map f xs"

value "map (λx. 2*x) [3::nat,2,5]"                    ⇝ [6, 4, 10]

lemma "sum (map (λx. 2*x) xs) = 2 * (sum xs)"
by (induct xs) auto
```

map recibe una función como argumento. La inducción es la de siempre, sobre la lista.

Resumen

Técnicas del tema

Técnica	Cuándo

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades
`induct n`	recursión sobre un natural
`induct xs`	recursión sobre una lista

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades
`induct n`	recursión sobre un natural
`induct xs`	recursión sobre una lista
`induct rule: f.induct`	la forma de la recursión de `f`

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades
`induct n`	recursión sobre un natural
`induct xs`	recursión sobre una lista
`induct rule: f.induct`	la forma de la recursión de `f`
`cases xs`	basta la forma, sin hipótesis de inducción

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades
`induct n`	recursión sobre un natural
`induct xs`	recursión sobre una lista
`induct rule: f.induct`	la forma de la recursión de `f`
`cases xs`	basta la forma, sin hipótesis de inducción
`arbitrary: ys`	el otro argumento cambia en la recursión

Técnicas del tema

Técnica	**Cuándo**
`also ... finally`	cadena de igualdades
`induct n`	recursión sobre un natural
`induct xs`	recursión sobre una lista
`induct rule: f.induct`	la forma de la recursión de `f`
`cases xs`	basta la forma, sin hipótesis de inducción
`arbitrary: ys`	el otro argumento cambia en la recursión
`quickcheck`	antes de demostrar: ¿es verdad?

- Recursión e inducción tienen la misma forma: un caso por constructor.

- Recursión e inducción tienen la misma forma: un caso por constructor.
- Se induce sobre la variable en la que recursa la función.

- Recursión e inducción tienen la misma forma: un caso por constructor.
- Se induce sobre la variable en la que recursa la función.
- Si la hipótesis de inducción no alcanza, generalice el enunciado.

- Recursión e inducción tienen la misma forma: un caso por constructor.
- Se induce sobre la variable en la que recursa la función.
- Si la hipótesis de inducción no alcanza, generalice el enunciado.
- `fix n`: «sea `n` arbitrario». Su papel exacto como regla lógica se verá en la Parte III.